



INTRODUCTION TO ADVANCED ROBOTICS

PROJECT 3

RECOGNISING OBJECTS

WHAT'S NEXT?

Project 1 taught the computer to measure a shape. Project 2 taught it to measure a **position** in millimetres. Both of those were things *you* told it how to look for.

This project is different in kind. You are not writing the rules any more. You are loading a model that somebody else trained on hundreds of thousands of labelled photographs, and asking it what it can see.

The rules now live in a file called `yolo_v5s.onnx`, twenty-nine megabytes of arithmetic. Your program's job is to feed it a picture, read the answer back out, and decide what to do with it.

This is the first project in the module where the interesting decisions are about trust rather than accuracy. A neural network will always answer, even when it is guessing. Learning to tell the difference is the real skill.

WHAT YOLO WILL NOT DO

Your shape detector worked on *any* triangle, square, circle or rectangle you could draw on a piece of paper. If it had four corners and straight edges, it found them.

The model in this project knows **eighty specific things** and nothing else. Hold up a hand-printed kite and it will not be recognised, because a kite is not one of the eighty. Hold up a pair of scissors and it will be.

That is not a flaw in the model. It is what "trained" means: somebody showed it pictures of eighty categories and labelled every one, and it learned those. It has never seen your project.

- *Before you write any code, look at the eighty class names on the next few slides and decide what you are actually going to put on the board. Choosing objects the model knows is the single biggest thing you can do to make this project work.*

TWO WAYS TO SPOT A CUP

The old way. You decide in advance what a cup looks like: round-ish, with a handle, taller than it is wide, sitting on a flat surface. You write each of those as a test. A blue mug with no handle fails all of them.

The new way. You show the computer ten thousand photographs of cups, labelled `cup`, and ten thousand of other things, labelled with what they actually are. It works out for itself which combination of shape, colour, texture and context means "cup". You never write a single test.

The second method is more accurate and far more flexible. It is also impossible to write by hand, impossible to read line by line, and completely incapable of telling you *why* it decided what it decided.

That last point is not a complaint, it is the deal. You get a strong answer with no reasoning attached. Learning to design around an answer you cannot interrogate is a skill in its own right, and it is the reason this project spends as long on thresholds as on maths.

WHAT IS YOLOV5?

YOLO is short for **You Only Look Once**. Older detection systems looked at the picture in stages — first find candidate regions, then classify each one. YOLO does the whole job in a single pass, which is why it is fast enough to run on a small computer beside a robot arm.

The `v5` is the fifth version. The `s` at the end of `yolov5s.onnx` means **small** — the fastest and least accurate of the family. There are larger models that are more accurate and too slow for this hardware. This one is the right trade.

Every part of that name matters when something goes wrong. `v5` tells you which set of output numbers to expect — read the wrong version's layout and your boxes land in nonsense places. `s` tells you how much accuracy you gave up for speed.

THE MODEL IS A FILE

This project does not train anything. Nobody at this station is going to teach a neural network anything from scratch. The learning already happened, months ago, on a large computer, and the result was saved to disk.

What you load is that saved result: `yolov5s.onnx`, about 29 megabytes. The `.onnx` extension is **Open Neural Network Exchange** — a neutral format, so a network trained in one framework can be used by OpenCV in another.

This is the part students forget. The file has to exist, in the right folder, on the machine that is running the program. The arm is a different computer from your

- *Copying your .py file across is not enough this time. The program looks for the model next to itself, so `yolov5s.onnx` and `coco.names` have to travel too. Get this wrong and the program tells you so by name.*

EIGHTY THINGS, AND ONLY THOSE

The model was trained on a dataset called **COCO**, which stands for Common Objects in Context. It contains photographs of everyday things, each labelled with what they are. Eighty of those labels are what this model knows.

They run from the obvious to the obscure. `person, bicycle, car, cup, scissors, teddy bear, toothbrush`. There is no "shape", no "block", and no "my object".

A useful classroom object is a laptop, a book, a bottle, a cup, a cellphone or a pair of scissors. Test your object against the list before you write anything.

A common and expensive mistake: spend an afternoon tuning thresholds on an object the model has never been trained to recognise. It will never work, and no threshold will save it. Check the list first — it takes ten seconds and it is free.

WHERE AM I?

```
script_dir = os.path.dirname(os.path.abspath(__file__))
```

A program does not know where it lives. It knows the folder you happened to be standing in when you ran it, and those are often not the same place.

`__file__` is the path to the script itself. `abspath` makes it absolute rather than relative, and `dirname` strips off the filename, leaving the folder. Read the names right to left and it nearly explains itself.

From that one line, the two file paths are easy:

```
model_path = os.path.join(script_dir, "yolov5s.onnx")  
labels_path = os.path.join(script_dir, "coco.names")
```

This is the difference between a program that works and one that works only on the machine that wrote it. `os.path.join` builds paths the right way on any system.

THE MARKERS GET A NEW JOB

You already taped two ArUco markers to the board in Project 2, and this project uses exactly the same two, in the same places, with the same detection code.

What changed is the question you ask of them. In Project 2 a marker meant **how far away is this**, measured in millimetres. Here it means **where does my workspace start and stop** — a rectangle on the screen, in plain pixels.

Same tool, new job. This is worth noticing, because in a real project you will keep meeting the same components doing different jobs, and the component does not change at all — only what you ask it.

- *Nothing about the detection code changes between Project 2 and Project 3. If your markers were found there, they will be found here. If they were not, no amount of work on the YOLO half will help you.*

WHY CROP THE PICTURE?

The camera sees your desk, your hands, the edge of the board, and half the room. The model was trained on photographs of objects, so a desk with a laptop on it is a strange input — and it will find the laptop anyway, but it will also find things on the desk you never meant to show it.

So the program uses the markers to decide what to look at. Everything outside the rectangle they define is thrown away before the model ever sees it.

This is a cheap, enormous win. The model gets a smaller, steadier, more consistent picture, which means better accuracy and fewer false positives — and it costs one crop. It also protects the numbers. A detection box reported on a cropped image is in cropped coordinates, so when you act on it you must remember which picture it came from. Know your coordinate system at the point you record a measurement, not later.

A PROBLEM YOU CAN SEE

Here is the fault that motivates half of this project. Put your hand in front of the camera and wave it slowly. The box drawn around your hand lags behind it.

The camera is not slow, and neither is the model. The problem is between them. A Linux camera driver works like this: it fills a buffer in the background, and when your program asks for a frame it gets **whatever is in that buffer at that moment** — not the newest picture.

By the time the model has finished thinking about frame 1, the camera has already produced frames 2, 3 and 4. Every frame you analyse is already out of date by the time you draw it.

- *Small delay, serious consequences. A sorter that is always half a second behind its hand puts the arm in the wrong place, every time, and no amount of threshold tuning will fix it.*

SOMEBODY HAS TO DRAIN THE BUFFER

The fix is to have a background thread whose only job is to read frames as fast as the camera produces them, throwing each one away except the newest. Your main loop then asks for whatever is currently in hand — which is always the most recent frame, seconds or milliseconds old rather than half a second.

One line sets it up. `daemon=True` means the thread is not allowed to keep the program alive on its own: when `main()` returns, Python takes the thread down with it instead of waiting forever.

```
self.thread = threading.Thread(target=self._update, daemon=True)
self.thread.start()
```

`target=self._update` names the function the thread runs. Starting the thread and defining what it does are two separate acts, in that order — the same shape as the vision pipeline you will build next.

SHARING A PICTURE BETWEEN TWO THREADS

Now two parts of the program can touch the same picture at the same moment: the background thread writing it, your main loop reading it. That is a race, and races do not fail politely.

The `with self.lock` block means "nobody else touches this while I am inside". Only one thread can hold the lock at a time, so the frame is either fully old or fully new — never half of each.

The `.copy()` on the way out is the other half of the answer, and it is the one people leave out. Without the copy you would be handed a reference to the very buffer the background thread is about to overwrite. You would start analysing a picture that changes underneath you, and the symptom would be boxes that shimmer for no reason anyone could explain.

- *The rule: hand out copies, never the original. It costs a few milliseconds and it removes an entire category of bug you would otherwise spend a day chasing.*

TURNING IT OFF POLITELY

Press `q` and the main loop breaks. If `release` just closed the camera, the background thread would still be blocked in a read on hardware that no longer exists. It has to be stopped first, and told to stop politely.

`self.running = False` is the flag the thread's own `while` loop is watching. It finishes the frame it is on, notices the flag, and exits at the top of the next pass.

`join(timeout=0.5)` then waits up to half a second for it to actually stop, and gives up rather than hanging forever if it will not.

```
self.running = False
if self.thread.is_alive():
    self.thread.join(timeout=0.5)
```

The `time.sleep(0.005)` inside the thread's loop is worth understanding too. Without it the thread would spin reading the camera flat out, burning CPU and heat while producing frames nobody wants. A tiny pause keeps it polite.

THE MODEL DOES NOT EAT PICTURES

A neural network cannot be handed a colour photograph. It expects a fixed-size grid of numbers, scaled a particular way, in a particular order, with the colour channels swapped. Getting any of those four things wrong produces a network that still runs and still answers — confidently and wrongly.

That preparation is one call:

```
blob = cv2.dnn.blobFromImage(  
    img, 1.0 / 255.0, self.input_size, [0, 0, 0], swapRB=True, crop=False  
)
```

The second argument divides every pixel by 255, turning 0–255 brightness into 0–1; the third resizes to the 640x640 the model was trained at.

The last two are the ones people miss. `swapRB=True` turns OpenCV's blue-green-red order into the red-green-blue the model expects; `[0, 0, 0]` subtracts nothing — zero says this model needs none. These four numbers are properties of *this* model, not your program — change one and boxes land wrong, quietly.

ASKING THE MODEL SOMETHING

```
self.net.setInput(blob)
outputs = self.net.forward(self.net.getUnconnectedOutLayersNames())
```

Two steps, and the first one surprises people. `setInput` does not run anything — it just hands the prepared picture to the network and waits. The network computes nothing at all until you call `forward`.

`getUnconnectedOutLayersNames` asks the network which of its outputs nothing else depends on, and returns just those. It saves work: you get the predictions without waiting for internal layers whose results you were never going to use.

The output is a NumPy array, not a list of Python objects. From here on you are reading numbers out of it, and the next slide is the map you need. One call to `forward` is one frame's worth of thinking — the most expensive line in the program, by a wide margin.

ONE ROW OF THE ANSWER

The model returns one row for every possible object position it considered — thousands of them, most of them describing nothing at all. Each row has the same layout, and it is worth writing it out:

Columns	Meaning	Units
row[0] to row[3]	cx, cy, w, h — centre and size	pixels in the 640 space
row[4]	Objectness — is there anything here	0 to 1
row[5] onwards	One score per class, 80 of them	0 to 1

Four numbers of geometry, then two separate confidence questions. `row[4]` asks "is there an object in this patch of picture at all", and the row of class scores asks "and if so, which one". A row can pass the first test and fail the second.

Every coordinate is in the model's own 640x640 space, not in pixels of your picture. That conversion is coming up, and forgetting it is the single most common reason boxes land nowhere near the objects.

THREE NUMBERS DECIDE WHAT IS REAL

```
def __init__(self, conf_thresh=0.45, score_thresh=0.5, nms_thresh=0.45):
```

Two gates and one filter, all set once in the constructor.

Name	Default	What it decides
conf_thresh	0.45	Is there an object in this patch at all
score_thresh	0.50	Is it this particular class
nms_thresh	0.45	How much overlap before a second box is discarded

NMS needs a picture to explain: the model draws several boxes per object, and NMS keeps the best of each cluster.

Raise it and real boxes vanish; lower it and you get three per cup.

One box can still be drawn where nothing is — confidently. NMS only removes duplicates, not mistakes; `conf_thresh` is your only lever.

DRAWING WHAT YOU FOUND

```
cv2.rectangle(annotated_img, (left, top), (left + width, top + height), (0, 255, 0), 2)
```

```
cv2.circle(annotated_img, (cx, cy), 4, (0, 0, 255), -1)
```

```
cv2.putText(annotated_img, label, (left, max(top - 5, 15)),  
            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)
```

A green rectangle, a blue centre dot, and the name and confidence printed above — readable enough to debug from across the room.

The dot is the centre point you would hand to the arm, and it exposes a mis-scaled box at a glance.

`max(top - 5, 15)` keeps the label on screen when a box sits near the top edge.

THE WHOLE PIPELINE ON ONE LINE

```
display_frame, log_info = vision.yolo_detect(cropped_frame)
```

Everything above is inside that one call, in this order:

- 1 Preprocess**
Resize, scale, swap channels, build the input blob.
- 2 Think**
One forward pass through the network.
- 3 Filter**
Two confidence gates on every row of the answer.
- 4 Convert**
Turn 640-space numbers back into real pixel coordinates.
- 5 Draw**
Rectangle, centre dot, name and confidence.

The lines before it in the loop crop to the workspace, so this sees just your board. Six steps, one call, about a hundred milliseconds.

STEP 1

Four packages this time, and one of them is new:

```
import cv2
import numpy as np
import time
import os
```

`cv2` and `numpy` you know. `time` still gives the arm its pauses. `os` is new, and it exists because this project has to find files on disk.

Now look at the line just below this TODO. `import threading` is already written for you — it is outside the `# ---` markers, so it is not yours to fill in. Read the starter before you start; some of the work is already done.

STEP 2

The robot import, unchanged from the last two projects:

```
from pymycobot.mycobot280 import MyCobot280
```

The vision half of this project needs no robot at all. That is worth saying out loud, because it means you can do every interesting thing in this project on a laptop with the arm switched off.

It also means the `try` block below will warn you about missing hardware and carry on, and that is the design working rather than something to fix.

Get the vision pipeline working before you worry about the arm. The interesting bugs in this project are all in the code you are about to write, and none of them need hardware to find.

STEP 3

Same robot startup as before, inside the `try` block:

```
print("Connecting to myCobot280...")
mc = MyCobot280('/dev/ttyAMA0', 1000000)
time.sleep(0.5)
mc.power_on()
time.sleep(0.5)
```

Nothing new here, which is the point. The port and the baud rate are the same, the pauses are the same, and the whole block still fails safely when there is no arm attached.

Notice which parts of this program *are* reusable. Every project in this module repeats the same four blocks — imports, robot startup, camera scan, display loop. Recognising them means you spend your attention on the part that is actually new.

STEP 4

Park the arm, same as always:

```
folded_angles = [0, 45, -90, -45, 0, 0]
print("Moving arm to initial folded position...")
mc.send_angles(folded_angles, 20)
time.sleep(2.0)
```

Folding the arm up and out of shot matters more in this project than in the last two. The model has been trained on photographs of rooms, and it is very good at finding objects in them. A shiny metal arm sitting in shot is an object in a room, and it will be detected as one.

If you see a box appear around the arm itself, this is why. It is not a bug in the model and there is no threshold that fixes it properly — the object is genuinely there. Move it out of the picture.

THREE FILES, NOT ONE

This is the one command that is different from the last two projects. The model and the label file have to arrive on the arm as well, because the program looks for them beside your script:

```
scp M2-P3-Starter.py yolov5s.onnx coco.names er@192.168.1.149:Documents
```

All three filenames go in one command. Forget the model and the program starts and then reports `YOLO Model Not Loaded` by name — which is a much friendlier failure than a traceback.

```
python M2-P3-Starter.py
```

Run it now, with the vision TODOs still empty. There is no error message this time — nothing happens, and that is worth understanding.

The crop returns nothing, the detector returns `"No Frame"`, and the display shows nothing.

A silent program is one whose guards all work. Fill the TODOs and the window appears.

STEP 5

The ArUco block, byte for byte the one from Project 2:

```
try:
    self.aruco_dict = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_6X6_250)
    self.aruco_params = cv2.aruco.DetectorParameters()
    self.detector = cv2.aruco.ArUcoDetector(self.aruco_dict, self.aruco_params)
    self.use_new_aruco = True
except AttributeError:
    self.aruco_dict = cv2.aruco.Dictionary_get(cv2.aruco.DICT_6X6_250)
    self.aruco_params = cv2.aruco.DetectorParameters_create()
    self.use_new_aruco = False
```

If you typed this carefully in Project 2, it is already in your muscle memory. Do not retype it from memory though — copy it across. One misplaced bracket here produces an `SyntaxError` that points at the wrong line and wastes an hour.

STEP 6

This TODO looks like one line but does not need to be. All it requires is the folder the script lives in:

```
script_dir = os.path.dirname(os.path.abspath(__file__))
```

```
model_path = os.path.join(script_dir, "yolov5s.onnx")  
labels_path = os.path.join(script_dir, "coco.names")
```

Nothing is opened or read yet — these are just strings naming where files should be. That comes in the next two steps, and keeping "decide where" separate from "read it" is what makes this code easy to test.

Check the folder now, before you write any more.

`yolov5s.onnx` and `coco.names` should be sitting next to `M2-P3-Starter.py` in your project folder. If they are not, get them now. On the `aim` the model must be in `Documents` beside your script. Copying only the `.py` across is the most common reason this project refuses to start — it costs one extra filename in the `scp` command.

STEP 7

Now load the network. It is one line, and it has a condition attached:

```
self.net = cv2.dnn.readNet(model_path) if os.path.exists(model_path) else None
```

The conditional expression — `A if test else B` — means "load the network if the file is there, otherwise set `net` to `None`". Same idea you used for the conditional in Project 1, written on one line.

It matters that it does not simply crash. A missing model file is an entirely likely mistake on a machine that has never run this project, and this line turns that mistake into a state the program can carry on from — and report by name.

The check further down — `if self.net is None: return img, "YOLO Model Not Loaded"` — is what turns that state into a message you can read. Defensive code is not paranoid; it is the difference between a bug report and a blank screen.

STEP 8

The label file is a plain text list, one class name per line, eighty lines long. Read it into a list:

```
self.classes = []  
if os.path.exists(labels_path):  
    with open(labels_path, "r") as f:  
        self.classes = [line.strip() for line in f if line.strip()]
```

Same defensive shape as Step 7: start empty, then fill it if the file exists. The program runs without labels rather than not at all.

`with open(...)` guarantees the file handle is closed afterwards, even if something inside goes wrong. Forget the `with` and you have a resource leak you will hardly notice.

The comprehension keeps each line, strips it, and skips blanks. `strip()` matters — a stray newline makes labels look right in the file and print with a space on screen.

STEP 9

First use of the markers in this project. Detect them and store the two centre points:

```
frame_contiguous = np.ascontiguousarray(frame, dtype=np.uint8)
try:
    gray = cv2.cvtColor(frame_contiguous, cv2.COLOR_BGR2GRAY)
```

`np.ascontiguousarray` packs the picture so OpenCV's fast routines accept it; then the same `if self.use_new_aruco` branch as Step 5, wrapped in a `try` because a half-hidden marker can throw.

```
if ids is not None and len(corners) >= 2:
    p1 = corners[0][0]
    p2 = corners[1][0]
    self.x1, self.y1 = int(p1[:, 0].mean()), int(p1[:, 1].mean())
    self.x2, self.y2 = int(p2[:, 0].mean()), int(p2[:, 1].mean())
    return True
```

Note `>= 2`, not `> 0`: two markers make a rectangle; one does not. The return value tells the caller whether the stored coordinates are trustworthy.

STEP 10

Now crop to the workspace. First enlarge, then cut:

```
fx, fy = 1.5, 1.5  
resized = cv2.resize(frame, (0, 0), fx=fx, fy=fy, interpolation=cv2.INTER_CUBIC)
```

The crop happens in the enlarged picture, so the marker coordinates from Step 9 are scaled by 1.5 to match.

`INTER_CUBIC` is a better-looking way to resize than the default, and it matters more than you would expect — a soft, blocky object is harder for the model to recognise.

```
margin = 25  
x_min, x_max = x_min + margin, x_max - margin  
y_min, y_max = y_min + margin, y_max - margin
```

The margin pulls the edges inward so the black marker borders stay outside the picture. Crop too tightly and the model sees a black frame around every object — which it may confidently identify as a picture frame or a window.

STEP 11

This TODO has no code of its own — it is the method that the next five TODOs fill in. Read the scaffolding you have been given:

```
if img is None or img.size == 0:
    return img, "No Frame"

if self.net is None:
    return img, "YOLO Model Not Loaded"

img_h, img_w = img.shape[:2]
```

Two guards and a measurement. `shape[:2]` gives height and width only, and you will need both on this slide's remaining steps to scale the boxes back.

The two guards are the ones from Steps 7 and 8 doing their job. Notice that they return `img` unchanged rather than `None` — so the caller still has a picture to display, with an honest message beside it instead of a blank window. `"No Frame"`, `"YOLO Model Not Loaded"` and `"Searching..."` — the three states; learn them on sight.

STEP 12

Preprocessing, already covered. Four settings, all properties of the model rather than of your program:

```
blob = cv2.dnn.blobFromImage(  
    img, 1.0 / 255.0, self.input_size, [0, 0, 0], swapRB=True, crop=False  
)  
self.net.setInput(blob)
```

`input_size` is (640, 640) — a *square*, set way back in the constructor. Your cropped workspace is almost certainly not square, and it does not need to be: the `resize` squashes it. The model does not know or care what your object's real proportions were.

That last point has a consequence worth stating plainly. If you squash a tall thin object into a square, it arrives distorted — and a sufficiently distorted object can fall outside what the model was trained on. Distant or badly framed objects fail here, and it looks like a detection problem rather than a framing one.

STEP 13

The forward pass — the one line that costs about a hundred milliseconds:

```
outputs = self.net.forward(self.net.getUnconnectedOutLayersNames())
```

Then two lines of tidying before you read anything:

```
predictions = outputs[0]
if len(predictions.shape) == 3:
    predictions = predictions[0]
```

The model returns its answer in batches, and you only ever send it one picture at a time. `[0]` takes the first — and only — batch, leaving a three-dimensional array: one row per candidate position.

The `if` is defensive again, and it earns its place. Library return shapes change between versions, and this program is already written to survive two different OpenCV layouts.

STEP 14

The longest TODO in the project, and the one worth slowing down for. Read the row layout again, then walk this loop:

```
for row in predictions:
    confidence = row[4]
    if confidence > self.conf_thresh:
        scores = row[5:]
        class_id = np.argmax(scores)
        if scores[class_id] > self.score_thresh:
```

Three rows are skipped, three more are skipped, and only then does a row become a detection. `row[4]` is objectness; `row[5:]` is every class score from index 5 onwards; `np.argmax` returns the position of the highest one, which is the class id as a plain integer.

Notice the gates are **nested**, not combined with `and`. That means a row failing the objectness test never even has its class scores examined — which is what you want, because it is the cheaper test and it discards almost everything.

STEP 14, CONTINUED

Inside both gates, four numbers become a box in real pixels:

```
cx, cy, w, h = row[0], row[1], row[2], row[3]
left = int((cx - w / 2.0) * x_factor)
top = int((cy - h / 2.0) * y_factor)
width = int(w * x_factor)
height = int(h * y_factor)
```

The model gives you a centre and a size. A rectangle needs a corner and a size, so subtracting half the width and height is how you convert — and it is the step people most often get wrong by half a box.

The `x_factor` and `y_factor` are the ratio between the model's 640-space and your actual cropped picture, computed before the loop. Both are separate because your image is not square, and using one factor for both squashes every box.

If every box is drawn but none land in the right place, the usual cause is skipping the scaling — numbers that look right but belong to a 640x640 picture that is not on screen.

STEP 15

```
indices = cv2.dnn.NMSBoxes(bboxes, confidences, self.score_thresh, self.nms_thresh)
```

One line, and it does the tidying. Note what is passed in: the boxes, the confidences that produced them, and two thresholds — the score threshold again, then the overlap threshold.

What comes back is a list of the boxes that survived:

```
if len(indices) == 0:  
    return img, "Searching..."  
  
if isinstance(indices, tuple) or len(indices.shape) > 1:  
    indices = indices.flatten()
```

Different versions return a tuple or array; `flatten` makes it indexable with an integer.

Returning "Searching..." keeps the terminal quiet while you hold an object the model does not know.

STEP 16

The last TODO inside the method. Turn surviving indices into a picture you can read:

```
for idx in indices:
    left, top, width, height = boxes[idx]
    cx, cy = left + width // 2, top + height // 2
```

One line worth pausing on. This one is long, and it is the only place in the project where the label file is actually consulted:

```
class_name = self.classes[class_ids[idx]] if class_ids[idx] < len(self.classes) else str(class_ids[idx])
```

Read it as a question: **if this id is inside the list of names, use the name; otherwise use the number itself.** A box labelled 37 instead of `toothbrush` means the labels did not load.

Then `label = f"{class_name} {confidences[idx]:.2f}"` builds the label and `detected_names.append(label)` collects it for the terminal. `.2f` is two decimal places — 0.8734 becomes 0.87.

STEP 17

The camera scan, unchanged since Project 1:

```
def find_working_camera(max_tests=5):  
    """Scans video indices and returns the first camera that produces frames."""  
    for idx in range(max_tests):  
        cap = cv2.VideoCapture(idx, cv2.CAP_V4L2)  
        if cap.isOpened():  
            ret, frame = cap.read()  
            cap.release()  
            if ret and frame is not None and frame.size > 0:  
                print(f"Found active camera at index {idx}")  
                return idx  
  
    return None
```

Worth noticing: this function opens a plain `cv2.VideoCapture` and immediately closes it, purely to test the index. The threaded camera you use for the rest of the program is a different object. Testing with one kind and running with another is normal, and worth naming so it does not confuse you later.

STEP 18

The biggest blank in the file — this TODO is the whole of `main()`, from the camera onwards. Build it in three pieces rather than all at once, and this is piece one:

```
cap_num = find_working_camera()
if cap_num is None:
    print("Error: No working camera stream found.")
    return

# Replace standard cv2.VideoCapture with ThreadedCamera
cap = ThreadedCamera(cap_num)
if not cap.isOpened():
    print("Error: Could not open camera.")
    return
```

Everything up to `while True:` is already written in the starter — the creation, the checks, and the object you will call `vision` on.

The point of the project sits in one line: `cap = ThreadedCamera(cap_num)` instead of `cv2.VideoCapture`.

The threaded camera is already written for you in the starter — your job is to use it and understand it.

STEP 19

Three small TODOs left, all inside the loop, and all one line each. This first one refreshes the workspace bounds:

```
vision.calculate_aruco_bounds(frame)
```

Called every frame, which sounds wasteful — the markers have not moved. It is cheap, and it is worth it, because it means the workspace follows the board if anybody nudges it. Cheap and every frame beats clever and once.

Then the crop, using whatever bounds the call above just found:

```
cropped_frame = vision.transform_frame(frame)
```

If the markers are not both visible, `calculate_aruco_bounds` returns `False` and leaves the old coordinates in place. `transform_frame` then falls back to the whole enlarged picture rather than failing. That fallback is deliberate: the model still works, it just sees more than it should.

STEP 20

The call that does the work:

```
display_frame, log_info = vision.yolo_detect(cropped_frame)
```

Same shape as every project in this module: two return values, unpacked into two names. One for the window, one for the terminal.

The terminal line below it is already written, and it is filtered the same way as before:

```
if "Searching" not in log_info and "No Frame" not in log_info:  
    print(f"\r[YOLOv5] {log_info}", end="")
```

Notice what is *not* filtered out. "YOLO Model Not Loaded" deliberately passes through this test and prints, because it is the one message you must never miss. Filtering on two specific strings leaves room for the rest to speak.

STEP 21

The final TODO. Show the picture:

```
if display_frame is not None and display_frame.size > 0:  
    cv2.imshow("YOLOv5 Object Detection", display_frame)
```

Identical in shape to Project 2, with a different window title. That title is the first thing to look for when the program seems to be doing nothing, so name it after what is in the window.

Then the shutdown, which for this project means stopping a thread as well as closing a camera:

```
cap.release()  
cv2.destroyAllWindows()
```

Because `cap` is a `ThreadedCamera`, `release` now stops the background thread before closing the camera — exactly the polite shutdown from the concepts section. `cv2.destroyAllWindows()` would not do it for you.

READING THE OUTPUT

```
status_msg = f"Detected: {' , '.join(detected_names)}"
```

One line listing everything currently on the board:

```
Detected: cup 0.87, scissors 0.52
```

Every box on screen appears here as a name and a number. `join` glues a Python list into one readable string with commas and spaces — the kind of small detail that makes a log worth keeping.

While nothing is recognised the line reads `Searching...`, and if the model is missing it reads `YOLO Model Not Loaded` and keeps going. Learn those two by sight.

The confidences are the whole subject of the next slide. A detection at 0.87 and one at 0.52 are both "Detected" — worth knowing the difference before you let an arm act.

BEFORE YOU TUNE ANYTHING

Symptom	Most likely cause	Check
YOLO Model Not Loaded	yolov5s.onnx not beside your script	Copy the model across too
Boxes labelled with numbers	coco.names missing or empty	Check Step 8 ran
Boxes in the wrong place	Boxes not scaled to the cropped image	Re-read the x_factor step
One object, several boxes	nms_thresh too high	Lower it, 0.45 to 0.3
Flickering on and off	conf_thresh too high for this object	Lower it and log the values
Box around the arm itself	The arm is in shot	Fold it out of the picture
Nothing, ever	Object is not one of the 80 classes	Read the label list

Five of those seven are mistakes you can make in the first ten minutes and check in the first ten seconds. Read the list before you tune anything — several of these symptoms get worse as you lower thresholds, which is exactly the wrong direction.

WHAT A THRESHOLD ACTUALLY COSTS

Before you change any number, write down what it costs you. There are three failure modes and they pull in different directions.

If you...	You will see	And you will lose
Lower <code>conf_thresh</code>	More detections overall	Real confidence in them
Raise <code>conf_thresh</code>	Cleaner, steadier output	Faint or unusual objects
Lower <code>nms_thresh</code>	One box per object	Two genuinely close objects

There is no setting that is right for every object on every board. The honest process is to choose numbers, write down what they got wrong, and change one at a time.

Changing three thresholds at once and finding that the output looks better tells you nothing about which change did it. Change one, retest, record. That is the whole

- *A model will always answer, even when it is guessing. The thresholds are not there to make it right — they are there to make it honest.*



EXTENSION 1

REJECT, DO NOT GUESS

THE BRIEF

Your program currently trusts every box it draws. A real machine cannot afford to act on a guess.

Add the discipline a detection system needs before it is allowed to move anything: a confidence gate, a rule about agreement over time, and a record of everything it refused.

- *A system that says “I am not sure” is far more useful to an engineer than one that never does. Your job is to earn the right to act.*

YOUR MISSION: REJECT, DO NOT GUESS

Turn a confident-looking detector into one you would let near an arm.

- 1 Set your own accept threshold**
Pick a confidence you are willing to act on and write down why. Then find the lowest number that still catches every object on your board without false positives. One number, defended in a sentence.
- 2 Demand agreement over time**
A single frame can lie. Require the same class, in roughly the same place, on two or three consecutive frames before you accept it. This is the single cheapest accuracy improvement available to you, and it is how real detection systems avoid acting on a flicker.
- 3 Log every rejection**
Record what you refused and why — low confidence, changed class, moved too far. Ten minutes of logs shows which objects the model finds hardest.
- 4 Hand it over**
Write the handover page: your accept threshold, your agreement rule, what the rejection log told you, and one improvement you would add.

- *Implement your gate in the decision logic, not by raising a threshold until the noise disappears. One hides the symptom; the other records the problem.*

YOUR DATA LOG

Record what the model said before you decide what to believe. Include the detections you rejected.

Object	Confidence	Accepted?	Why / what happened
cup			
scissors			
cellphone			
my printed shape			

Then write your acceptance rule before you code it:

```
ACCEPT if confidence >= _____ AND same class on _____ frames
REJECT if class changed from _____ to _____
OTHERWISE _____
```

- *A rule you cannot say out loud is a rule you have not decided yet.*

DEFINITION OF DONE

- ☐ The model loaded from a file sitting beside your script, with no error messages on screen
- ☐ Workspace cropped between both markers, from three different camera positions
- ☐ At least three different objects from the 80 classes detected, named correctly, from several distances
- ☐ Your accept threshold chosen, written down, and defended in one sentence
- ☐ Rejection log collected over ten minutes of running, and read

GO FURTHER

- Track position over time. The centre dot already gives you a coordinate. Store a few frames of it and you have measured how much your detection jitters — which is the number that decides whether an arm can safely act on it.
- Compare the small model against a larger one. The larger model is more accurate and too slow here. Find out how much too slow, and where the trade-off would change if the arm were not using the CPU for anything else.
- Report every class, not just the best one. Right now you keep the highest scoring class per detection. Show the runner-up too and you can see how close a decision actually was.
- Make it work on a different board. Swap in the laptop, the water bottle, the book. Which ones work without any code change, and which needed a threshold you would rather not have set?

MISSION DEBRIEF

1. You lower `conf_thresh` and get more detections. How would you tell a real new object from a false positive, using only the numbers on screen?
2. The model returns a box around a `toothbrush` with a confidence of 0.46, just over your threshold. Should the arm move? Defend your answer.
3. Why does the program scale boxes with two separate factors, `x_factor` and `y_factor`, instead of one?
4. This project could tell you what something is but not how far away it was. What would you have to bring back from Project 2 to close that gap?